

A Comparison of Pair Versus Solo Programming Under Different Objectives: An Analytical Approach

Author(s): Milind Dawande, Monica Johar, Subodha Kumar and Vijay S. Mookerjee

Source: *Information Systems Research*, Vol. 19, No. 1 (March 2008), pp. 71-92

Published by: INFORMS

Stable URL: <http://www.jstor.org/stable/23015422>

Accessed: 10-11-2016 12:53 UTC

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at
<http://about.jstor.org/terms>



INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Information Systems Research*

A Comparison of Pair Versus Solo Programming Under Different Objectives: An Analytical Approach

Milind Dawande

School of Management and the School of Computer Science, University of Texas at Dallas,
Richardson, Texas 75083, milind@utdallas.edu

Monica Johar

The Belk College of Business, University of North Carolina at Charlotte,
Charlotte, North Carolina 28223, msjohar@uncc.edu

Subodha Kumar

Michael G. Foster School of Business, University of Washington, Seattle, Washington 98195,
subodha@u.washington.edu

Vijay S. Mookerjee

School of Management, University of Texas at Dallas, Richardson, Texas 75083, vijaym@utdallas.edu

This study compares the performances of pair development (an approach in which a pair of developers jointly work on the same piece of code), solo development, and mixed development under two separate objectives: effort minimization and time minimization. To this end, we develop analytical models to optimize module-developer assignments in each of these approaches. These models are shown to be strongly NP-hard and solved using a genetic algorithm. The solo and pair development approaches are compared for a variety of problem instances to highlight project characteristics that favor one of the two practices. We also propose a simple criterion that can reliably recommend the appropriate approach for a given problem instance. Typically, for efficient knowledge sharing between developers or for highly connected systems, the pair programming approach is preferable. Also, the pair approach is better at leveraging expertise by pairing experts with less skilled partners. Solo programming is usually desirable if the system is large or the effort needed either to form a pair or to code efficiently in pairs is high. Solo programming is also appropriate for projects with a tight deadline, whereas the reverse is true for projects with a lenient deadline. The mixed approach (i.e., an approach where both the solo and pair practices are used in the same project) is only indicated when the system consists of groups of modules that are sufficiently different from one another.

Key words: extreme programming; software development methodology; pair programming; integer programming; genetic algorithms; heuristics

History: Paulo Goes, Senior Editor; Giri Kumar Tayi, Associate Editor. This paper was received on April 7, 2006, and was with the authors 8 months for 2 revisions.

1. Introduction

The widespread adoption of information technology over the last few decades has created new challenges of delivering information system solutions under ever-tightening deadlines. Development processes, however, remain challenging and labor intensive and hence, new development practices continue to be proposed. These new practices are usually difficult to evaluate; thus claims made by the proponents of new approaches often go untested. This is partly because software, being largely intangible, resists quantitative analysis and measurement.

Another reason that retards analytic study is the perception among software professionals that because software development is a creative process, it should not be organized and managed.

1.1. Problem and Motivation

In 1995, Kent Beck, Ward Cunningham, and Ron Jefferies began to explore the extremes of certain software development practices in an attempt to keep code clean and simple, and ensure flexibility when confronted with changing requirements (Astels et al. 2002). They proposed a novel software development

approach, termed *extreme programming* (XP), where two developers simultaneously work on the same piece of code (e.g., a module, function, unit, etc.) (Beck 2000). Typically, one member of the pair (called the *driver*) actually writes the code, while the other (called the *navigator*) observes the creation of the code, suggests improvements in structure, points out tactical and strategic defects, etc. (Williams and Kessler 2003). It has been argued that although pair programming may increase the effort to develop a piece of code as compared to solo programming, this extra effort is often compensated by lower system integration and testing effort. Several studies have reported that pair programming may lower the integration and testing effort by 40% to 60% (Erdogmus and Williams 2003, Kuppuswami et al. 2003), but a more conservative figure of 15% has also been reported (Williams and Kessler 2003). Our model draws from several experimental studies to derive an expression for the reduction in system integration effort resulting from the use of pair programming. The key trade-off in our model is between the extra effort needed in pair programming to develop modules and the lower effort needed to integrate these modules to create a working system.

In addition to pair programming, XP has several other recommendations, including pair splitting, test planning before coding, and continuous integration (Beck 2000). Pair splitting ensures that knowledge about the system is disseminated among the developers. By pairing with many different partners, developers learn more about the system's architecture, and share programming skills (style, tools, techniques, etc.) with other developers. Test data is written even before coding is begun and the system is tested and integrated almost continuously. Finally, each team member is allowed to make changes to any part of the system.

An XP project also entails continuous and intense user involvement. Users pick valuable system features (called *user stories*) that describe a path through the system (Astels et al. 2002). A user story serves the same purpose as a use case in traditional software development (Wells 2003). During an *iteration*, a set of stories is chosen for development. An iteration can run for as long as six weeks, but a recent survey showed that two weeks and three weeks are the most popular durations (Beck and Fowler 2001). Iterations

occur in a sequence that depends on several factors including the functionality delivered by the iteration, and the cost and speed of development. Within an iteration, user stories are further broken down into specific programming tasks (referred to here as *modules*) and pairs of developers are then assigned to these tasks (Wells 2003).

As is clear from the above discussion, there are many aspects embedded in the overall XP methodology. Not all aspects, however, are found in every XP project. In this paper, our analysis focuses on two of the most striking features, namely, pair programming and pair splitting. In practice, the task of assigning modules to developer pairs is typically done without careful analysis (Williams and Kessler 2003). However, one must address this issue if the numerous benefits of pair programming and pair splitting are to be realized.

1.2. Objective and Contributions

This study compares the performances (based on effort and time) of pair and solo programming. Because the performance of an approach is affected by the quality of module-developer assignments, any statement about the relative performance of the two approaches must be made for a set of good quality assignments. This limits the use of empirical data, at least until the pair programming practice matures and the knowledge of this approach better assimilates in the software industry. To further complicate matters, the use of an empirical approach to compare performance hinges on the ability to obtain data on similar projects developed using the pair and solo approaches.

Given these limitations, we follow an analytical approach to gain insights on the performance of the pair approach relative to the solo approach. Two mathematical models are developed to find optimal module-developer assignments for pair and solo programming. The goal in the effort minimization model is to minimize the total effort (measured in person-weeks) needed to develop a system on or before a specified deadline, whereas the goal in the time minimization model is to minimize the time needed to develop a system while respecting an effort constraint. For both objectives, it is extremely difficult to solve realistic problem instances to optimality using state-of-the-art solvers (such as CPLEX). Hence, we

propose a heuristic technique—based on a genetic algorithm (GA)—to solve the problem.

For the effort minimization model, without the deadline constraint, the objective function can be minimized by assigning a single pair of developers (or a single developer for solo programming) to all development tasks. This way, the assigned pair (developer) acquires maximum knowledge of the system and the integration effort is the lowest possible. However, a single pair (developer) also takes a long time to complete the work; thus, in the interest of the deadline, the different development tasks must be done in parallel by multiple pairs (developers). For the time minimization model, the presence of an effort constraint restricts the number of pairs (developers) that can be used, making the task of producing good quality module-developer assignments quite challenging. We next summarize the main contributions of this work.

We propose a simple criterion that allows us to predict whether a particular project can be completed with less effort using pair programming, or whether the traditional practice of solo programming should be preferred. This criterion is derived using a homogeneous system approximation—a simplified approximation of the system where detailed system parameters (e.g., the effort to develop individual modules, number of connections a module has with other modules, etc.) are replaced by a single mean value that applies to the whole system. The homogeneous approximation is also used to investigate whether the practice of mixing development regimes for the same project can lower development effort over one of the pure approaches (solo or pair).

To extend our analytical results with numerical simulations, we develop a GA that is shown to provide optimal or near-optimal solutions for modest-sized problems. For problems where a comparison with the optimal solution cannot be made, the gap between a lower bound on the optimal solution and the solution provided by the GA is shown to be reasonably small. The GA is therefore used to solve many realistic problems that are generated based on a real system being developed using pair programming at a telecommunications software company. An experiment is conducted to investigate the factors that explain performance differences between the pair and solo development methods. The two development

regimes are compared with one another under two different objective criteria: effort and time. Typically, when knowledge sharing between developers is efficient or when the system is highly connected (with many modules that are functionally interdependent), the pair programming practice is preferable. On the other hand, the solo programming practice is usually desirable if the effort needed either to form a pair or to code efficiently as a pair is high. The solo programming practice also emerges more appropriate for projects with a tight deadline, while the reverse is true when the deadline is sufficiently lenient.

1.3. Literature Review

We examine several important principles of XP: pair programming, knowledge sharing, user involvement, and testing and debugging. One of the most novel features of XP is pair programming. The claimed benefits of pair programming are: better code quality, shorter cycle time, happier developers, better trust and teamwork, more knowledge transfer, and enhanced learning. (Ambler 2002, Astels et al. 2002, Williams and Kessler 2003). Some of these benefits have been experimentally verified: reduced total effort (Williams and Kessler 2003), fewer unit and integration errors (Cockburn and Williams 2000, Williams et al. 2000), and simpler code (Wood and Kleb 2002). XP also recommends pair splitting—a practice in which developers change partners during the project. Pair splitting reduces the training time to assimilate new members, distributes the training burden across the team (Shukla 2002, Williams and Kessler 2003), and maintains productivity in an environment with high personnel turnover (Benedicenti and Paranjape 2001).

Better knowledge sharing among developers is a key benefit of pair programming (Williams and Kessler 2003). Pairing developers allows them to share knowledge and form a common understanding of the system and the development tasks. Sharing occurs in several areas such as the client's evolving requirements and business environment, new hardware, development tools and languages, and evolving technologies (Curtis et al. 1988, Waltz et al. 1993). Sharing of knowledge, however, does not only take place by being taught or instructed, but also by *becoming* a practitioner (Brown and Duguid 1991). This tacit or implicit notion of knowledge sharing is at the heart of

pair programming. More explicit knowledge sharing methods, however, also contribute to knowledge sharing. Dingsoyr (2002) finds that the use of knowledge management tools benefits software quality, reduces development costs, and improves developer morale.

Closely tied to facilitating knowledge sharing in a software project is the need to coordinate the efforts of the members of a development team. One way to facilitate coordination is to reduce the cost of moving information between developers. Cockburn and Highsmith (2001) state that in XP such cost reduction is achieved by placing people physically closer to one another, replacing documents with face-to-face discussions, and by improving the team's amity so that members are more inclined to relay valuable information quickly. Meixell et al. (2006) apply a coordination-theoretic perspective to determine the number of developers and the number of sub-tasks into which any given task of a project should be divided.

Coordination is not only needed among developers, but also between developers and end users. Involving end users in the development process increases the likelihood that requirements are correctly derived, leading to more useful and usable systems (Baecker et al. 1995, Nielsen 1993). Although several methods for deriving software requirements have been proposed (Ippolito and Murman 2001), all these methods must contend with requirements that frequently change during development (Grunbacher and Hofer 2003). Agile software development methods, such as XP, through continuous end user involvement, are claimed to better deal with evolving requirements (Beck 2000).

Finally, we discuss system testing and integration practices within XP. This task involves ensuring that the components of a software system interact without error. Pressman (1992) observes that, irrespective of the specific development model followed, system testing and integration is required for successfully executing any project. The increasing complexity of software products, together with shortened development cycles and higher customer expectations of quality, has elevated the role of software integration and testing (Hailpern and Santhanam 2002). System integration and testing can be done in a planned or an ad hoc manner. Traditionally, system testing and

integration is often planned or controlled (Pressman 1992). XP, on the other hand, proposes continuous and ad hoc integration, i.e., the system is fully integrated at all times and XP teams typically "build" the system several times each day on an "as needed" basis. A claimed benefit of such fine-grained integration and testing is that it allows easy backtracking and defect isolation (Beck 2000).

The rest of this paper is organized as follows. Section 2 presents the effort minimization and the time minimization models together with a GA devised to solve these models. In §3, we derive a criterion that can help choose between the pair and solo approaches on the basis of development effort. We also estimate regression models that provide a preliminary understanding of the factors that affect the relative performance of the pair and solo approaches. Section 4 details a series of controlled experiments conducted to closely examine the impact of these factors on the performances of the pair and solo approaches. Section 5 discusses the implications, limitations, and possible extensions of the study, and concludes the paper.

2. Model and Heuristic Solution

In this section, we present optimization models for the effort minimization problem and the time minimization problem. At the end of this section, we propose a GA to solve these optimization models. Given that pair programming is relatively new, we contacted several well-known XP practitioners¹ to improve our understanding of how pair programming occurs in practice.

2.1. Preliminaries

For both the effort minimization and time minimization problems, we present three versions of the model: *solo*, *pair*, and *mixed*. We use the term *module* to refer to a unit of work assigned to a developer or a pair of

¹ Private Communications: E. Herman, Product Sight Corporation, Bellevue, WA.; R.C. Martin, CEO, President and Founder of Object Mentor Inc., Gurnee, IL.; C. Poole, Founder and Principal of Poole Consulting, Poulsbo, WA.; B. Ramsdell, Co-founder, Brute Squad Labs., acquired by Sendmail, Inc., Emeryville, CA.; R. Rangan, CTO of Product Sight Corporation, Bellevue, WA.; A. Ridlehoover, SBI Group, Bellevue, WA. S. Sidhartha, Microsoft Corporation, Redmond, WA.

developers. The solo model requires that each module be assigned to a single developer; a developer can be assigned to more than one module. The pair version requires that each module be assigned to a pair of developers; the same pair can, of course, work on multiple modules. In the mixed model, a module can be assigned to a single developer or a pair of developers.

We consider a system that consists of m modules that are divided into G groups. In keeping with common XP practice, a group of modules corresponds to some functionality that must be developed and delivered to users before development work on the next group is begun. Thus we require that the groups be developed sequentially, although the modules within a group can be developed in any convenient sequence. In each version of the model, the total *system development effort* (measured in person-weeks) is the sum of the *system module development effort*, the *system pair formation effort* (equals zero for solo), and the *system integration effort*.

2.1.1. Module Development Effort. The module development effort is the effort needed to develop all the modules in the system. For all three regimes (i.e., solo, pair, and mixed), the module development effort depends on the characteristics of the developers (such as *expert*, *average*, or *novice*) involved. The module development effort for any given module depends on whether it is developed by a single developer or a pair of developers. This effort is typically higher for the pair approach (relative to solo development) and has been attributed to the additional effort needed to communicate between the members of a pair (Beck 2000). This higher effort may be partially diminished by the additional documentation effort that often accompanies the solo approach to ensure formal communication within the team. However, a *net* higher development effort associated with pair development (referred to here as the *pair development overhead*) has been consistently observed in experimental studies (Williams et al. 2000, Nosek 1998, Nawrocki and Wojciechowski 2001, Lui and Chan 2004). The pair development overhead has several positive effects: increased knowledge sharing (Beck 2000), superior testing and quality of code (Succi et al. 2001), and better compatibility of the code with the rest of the system (Highsmith and Cockburn 2001). Thus while

pair development requires additional module development effort, its use could reduce the system integration effort and hence, the *total* system development effort.

2.1.2. Pair Formation Effort. The second component of the total system development effort is the system pair formation effort. Clearly this component is zero for solo programming. The system pair formation effort is the sum of the pair formation efforts incurred each time a unique pair is formed. The pair formation effort is a one-time effort incurred by a pair of developers to establish the mutual understanding needed to work effectively as a team. During pair formation, developers learn to give and accept objective suggestions, and to communicate during development. For a pair, this effort could change with the characteristics of the developers (such as *expert*, *average*, or *novice*) involved in the pair (Williams and Kessler 2003).

2.1.3. System Integration Effort. The third component of the total system development effort is the system integration effort. This effort is calculated as the sum of the efforts needed to integrate each pair of functionally dependent modules (or links) in the system. To calculate the system integration effort, we use results from pair programming experiments that have noted several important observations: (1) Pair programming typically lowers system integration effort (Erdogmus and Williams 2003); (2) this reduction is found to increase with the extent to which pair programming is used in the project (Kuppuswami et al. 2003); and (3) the total reduction from knowledge sharing in pair programming is found to be directly related to the complexity of the integration tasks (Cockburn and Williams 2000). The above findings lead to an expression for system integration effort as discussed below.

Rather than directly propose an expression for the system integration effort for a project as a whole, we arrive at an expression for this effort in a constructive manner by first developing an expression at the link level. Let λ_{iz}^s be the effort needed to integrate modules i and z using the solo approach when *no common developers* are used in the development of these modules. Similarly, λ_{iz}^p denotes the corresponding effort when both modules are developed using the pair approach, and λ_{iz}^0 denotes the integration effort when

one module is developed using the solo approach and the other using the pair approach.

Let $C_{izj} = 1$ if developer j is common to modules i and z ; 0 otherwise. The total *reduction* in integration effort (R_{iz}) should be directly proportional to the number and characteristics of the developers that are common (based on (2) above) and the no-common-developer integration effort (based on (3) above). This total reduction can be expressed as the sum of the reductions due to each common developer; the reduction due to a particular common developer j is $R_{izj} = \alpha_j \lambda_{iz}^k$, where $k = s, p$, or Ω , depending on whether modules i and z are developed using the solo, pair, or mixed approach, respectively. Here α_j (referred to here as the *knowledge sharing coefficient* for developer j) is a proportionality constant that captures the impact of developer j being common to the development of the two modules. The total reduction in integration effort for integrating modules i and z is, therefore, $R_{iz} = \sum_{\{j: C_{izj}=1\}} R_{izj} = \lambda_{iz}^k \sum_j C_{izj} \alpha_j$, where $k = s, p$, or Ω . The resulting integration effort for a link is, therefore, $\lambda_{iz}^k - R_{iz} = \lambda_{iz}^k (1 - \sum_j C_{izj} \alpha_j)$.

The value of α_j for a given project is influenced by factors such as the use of knowledge sharing tools in

the project, developer experience, and the use of collaborative methods (Dingsoyr 2002). The experience gathered from previous assignments completed by developer j can be used to estimate the value of this coefficient for that developer. Clearly, for the integration effort to be nonnegative, α_j should be such that $\sum_j C_{izj} \alpha_j \leq 1$. Note that this is not a model constraint, but a constraint on the feasible values that can be chosen for the knowledge sharing coefficient. Table 1 provides a detailed list of model parameters and their definitions.

Additional Variable Definitions

$C_{izj} = X_{ij}X_{zj} = 1$ if developer j is common to modules i and z ; 0 otherwise

$\beta_{ijl} = X_{ij}X_{il} = 1$ if module i is assigned to developer pair (j, l) ; 0 otherwise,

$n_{jl} = \sum_{i=1}^m \beta_{ijl}$; n_{jl} is the number of modules assigned to the developer pair (j, l) ,

$q_{jl} = 1$ if $n_{jl} \geq 1$; 0 otherwise,

$w_{iz}^s = 1$ if both modules i and z are developed using the solo approach; 0 otherwise,

$w_{iz}^p = 1$ if both modules i and z are developed using the pair approach; 0 otherwise, and

Table 1 Model Parameters and Decision Variables

Symbol	Definition	Remarks
G	Number of groups	
m	Number of modules in the system	
M_g	Set of modules in group g	
S	Number of developers	
α_j	Knowledge sharing coefficient for developer j	<i>A high value means more effective knowledge sharing. Incurred by the pair to establish mutual understanding to work effectively as a team.</i>
f_{jl}	Effort required to form the developer pair (j, l)	
D_{ij}^s	Development effort for module i if developed by a single developer j	<i>Overhead incurred to collaborate and code as a pair. These values specify the connectivity of the modules in the system.</i>
D_{ijl}^p	Development effort for module i if developed by a developer pair (j, l)	
σ	Pair development overhead = $2D_{ijl}^p / (D_{ij}^s + D_{il}^s)$	
p_{iz}	= 0 if modules i and z are not connected; = 1 otherwise	
L	Link Density = $2 \sum_{i=1}^m \sum_{z=i+1}^m p_{iz} / m(m-1)$	<i>The link density is the ratio the number of links in the system to the maximum possible links.</i>
N_g	Set of links with one module in group g and the other module in a group with index at most g .	
λ_{iz}^k	Effort to integrate modules i and z when there are no common developers between these modules; $k = s, p$, or Ω , depending on whether modules i and z are developed using the solo, pair, or mixed approach, respectively.	<i>To reflect the impact of superior quality (lower integration effort) resulting from pair development, we expect $\lambda_{iz}^s \geq \lambda_{iz}^\Omega \geq \lambda_{iz}^p$</i>
T	Desired duration of the project (deadline)	<i>Weeks</i>
B	Effort budget for the project	<i>Person-weeks</i>
X_{ij}	= 1 if developer j is assigned to module i ; = 0 otherwise	<i>Decision variable</i>

$w_{iz}^{\Omega} = 1$ if one of modules i and z is developed using the solo approach and the other is developed using the pair approach; 0 otherwise.

2.2. Effort Minimization

The goal is to select an optimal module-developer assignment scheme such that the total system development effort is minimized. In addition, it is required that the project be completed on or before a specified deadline. The formal model for effort minimization is presented below.

$$\begin{aligned} \text{Minimize } & \sum_{i=1}^m \sum_{j=1}^S D_{ij}^s X_{ij} + \sum_{i=1}^m \sum_{j=1}^S \sum_{l=j+1}^S \beta_{ijl} (D_{ijl}^p - D_{ij}^s - D_{il}^s) \\ & + \sum_{j=1}^{S-1} \sum_{l=j+1}^S f_{jl} q_{jl} + \sum_{i=1}^{m-1} \sum_{z=i+1}^m \left[\lambda_{iz}^s w_{iz}^s + \lambda_{iz}^p w_{iz}^p + \lambda_{iz}^{\Omega} w_{iz}^{\Omega} \right. \\ & \left. - \sum_{j=1}^S \alpha_j (\lambda_{iz}^s w_{iz}^s C_{izj} + \lambda_{iz}^p w_{iz}^p C_{izj} + \lambda_{iz}^{\Omega} w_{iz}^{\Omega} C_{izj}) \right] p_{iz}. \end{aligned}$$

Subject to:

Time Constraints

$$\begin{aligned} & \sum_{i \in M_1} \left[D_{ij}^s X_{ij} + \sum_{l \neq j} \beta_{ijl} (0.5 D_{ijl}^p - D_{ij}^s) \right] + \sum_{l=1}^S 0.5 f_{jl} q_{jl} \\ & + \sum_{(i-z) \in N_1} \left[\frac{\lambda_{iz}^s}{2} (w_{iz}^s x_{ij} + w_{iz}^s x_{zj}) + \frac{\lambda_{iz}^p}{4} (w_{iz}^p x_{ij} + w_{iz}^p x_{zj}) \right. \\ & \left. + \frac{\lambda_{iz}^{\Omega}}{3} (w_{iz}^{\Omega} x_{ij} + w_{iz}^{\Omega} x_{zj}) \right. \\ & \left. - \sum_{l=1}^S \alpha_l \left(\frac{\lambda_{iz}^s}{2} (w_{iz}^s x_{ij} C_{izl} + w_{iz}^s x_{zj} C_{izl}) \right. \right. \\ & \left. \left. + \frac{\lambda_{iz}^p}{4} (w_{iz}^p x_{ij} C_{izl} + w_{iz}^p x_{zj} C_{izl}) \right. \right. \\ & \left. \left. + \frac{\lambda_{iz}^{\Omega}}{3} (w_{iz}^{\Omega} x_{ij} C_{izl} + w_{iz}^{\Omega} x_{zj} C_{izl}) \right) \right] \leq T_1 \\ & \forall j = 1, 2, \dots, S \end{aligned}$$

$$\begin{aligned} T_1 + & \sum_{i \in M_2} \left[D_{ij}^s X_{ij} + \sum_{l \neq j} \beta_{ijl} (0.5 D_{ijl}^p - D_{ij}^s) \right] \\ & + \sum_{(i-z) \in N_2} \left[\frac{\lambda_{iz}^s}{2} (w_{iz}^s x_{ij} + w_{iz}^s x_{zj}) + \frac{\lambda_{iz}^p}{4} (w_{iz}^p x_{ij} + w_{iz}^p x_{zj}) \right. \\ & \left. + \frac{\lambda_{iz}^{\Omega}}{3} (w_{iz}^{\Omega} x_{ij} + w_{iz}^{\Omega} x_{zj}) \right. \\ & \left. - \sum_{l=1}^S \alpha_l \left(\frac{\lambda_{iz}^s}{2} (w_{iz}^s x_{ij} C_{izl} + w_{iz}^s x_{zj} C_{izl}) \right. \right. \end{aligned}$$

$$\begin{aligned} & + \frac{\lambda_{iz}^p}{4} (w_{iz}^p x_{ij} C_{izl} + w_{iz}^p x_{zj} C_{izl}) \\ & \left. + \frac{\lambda_{iz}^{\Omega}}{3} (w_{iz}^{\Omega} x_{ij} C_{izl} + w_{iz}^{\Omega} x_{zj} C_{izl}) \right) \leq T_2 \\ & \forall j = 1, 2, \dots, S. \end{aligned}$$

Similarly, there are sequential constraints for groups $3, 4, \dots, G-1$.

Finally,

$$\begin{aligned} T_{G-1} + & \sum_{i \in M_G} \left[D_{ij}^s X_{ij} + \sum_{l \neq j} \beta_{ijl} (0.5 D_{ijl}^p - D_{ij}^s) \right] \\ & + \sum_{(i-z) \in N_G} \left[\frac{\lambda_{iz}^s}{2} (w_{iz}^s x_{ij} + w_{iz}^s x_{zj}) + \frac{\lambda_{iz}^p}{4} (w_{iz}^p x_{ij} + w_{iz}^p x_{zj}) \right. \\ & \left. + \frac{\lambda_{iz}^{\Omega}}{3} (w_{iz}^{\Omega} x_{ij} + w_{iz}^{\Omega} x_{zj}) \right. \\ & \left. - \sum_{l=1}^S \alpha_l \left(\frac{\lambda_{iz}^s}{2} (w_{iz}^s x_{ij} C_{izl} + w_{iz}^s x_{zj} C_{izl}) \right. \right. \\ & \left. \left. + \frac{\lambda_{iz}^p}{4} (w_{iz}^p x_{ij} C_{izl} + w_{iz}^p x_{zj} C_{izl}) \right. \right. \\ & \left. \left. + \frac{\lambda_{iz}^{\Omega}}{3} (w_{iz}^{\Omega} x_{ij} C_{izl} + w_{iz}^{\Omega} x_{zj} C_{izl}) \right) \right] \leq T \\ & \forall j = 1, 2, \dots, S. \end{aligned}$$

Module Assignment Constraints

For Solo Programming

$$\sum_{j=1}^S X_{ij} = 1 \quad \forall i = 1, 2, \dots, m,$$

exactly one developer is assigned to every module.

For Pair Programming

$$\sum_{j=1}^S X_{ij} = 2 \quad \forall i = 1, 2, \dots, m,$$

exactly two developers are assigned to every module.

For Mixed Programming

$$\sum_{j=1}^S X_{ij} \geq 1 \quad \forall i = 1, 2, \dots, m,$$

all modules must be assigned to at least one developer.

$$\sum_{j=1}^S X_{ij} \leq 2 \quad \forall i = 1, 2, \dots, m,$$

at most two developers can be assigned to any module.

In the objective function, the term $\sum_{i=1}^m \sum_{j=1}^S D_{ij}^s X_{ij} + \sum_{i=1}^m \sum_{j=1}^S \sum_{l=j+1}^S \beta_{ijl} (D_{ijl}^p - D_{ij}^s - D_{il}^s)$ measures the total module development effort. For a module i , $(D_{ijl}^p - D_{ij}^s - D_{il}^s)$ measures the extra development effort incurred when module i is developed by a pair of developers. The term $\sum_{i=1}^m \sum_{j=1}^S D_{ij}^s X_{ij}$ represents the total development effort using solo programming for all the modules, and the term

$$\sum_{i=1}^m \sum_{j=1}^S \sum_{l=j+1}^S \beta_{ijl} (D_{ijl}^p - D_{ij}^s - D_{il}^s)$$

represents the total pair development overhead for modules that were developed by the pair approach.

The second term in the objective function, $\sum_{j=1}^{S-1} \sum_{l=j+1}^S f_{jl} q_{jl}$, measures the total pair formation effort. The indicator variable q_{jl} is set to 1 if the number of modules assigned to developer pair (j, l) is at least one. The last term in the objective function measures the integration effort; the justification for this expression has already been provided earlier.

We next discuss the formulation of the project deadline constraint. Note that we require that the groups in the system be completed sequentially. The project completion time constraint enforces that the last group must be completed before (or by) the deadline T . The completion time of the last group is calculated as the completion time of the last-but-one group plus the time required for developing and integrating the last group. In general, the completion time of any group is calculated as the completion time of the previous group plus the development and integration time required for this group. The time required for a group is the maximum of the values of the time spent by each developer in that group. For any group, if module i is developed by a single developer j , then this developer incurs a development time of D_{ij}^s . If, on the other hand, module i is developed by a pair (j, l) , then developers j and l each incur a development time of $0.5D_{ijl}^p$. Over all the modules in group g , the total module development time for developer j is, therefore, $\sum_{i \in M_g} D_{ij}^s X_{ij} + \sum_{l \neq j} \beta_{ijl} (0.5D_{ijl}^p - D_{ij}^s)$. Similarly, if a developer pair (j, l) is formed, then developers j and l each incur a pair formation time of $0.5f_{jl}$. Hence, the total pair formation time for developer j is $\sum_{l=1}^S 0.5f_{jl} q_{jl}$. We next consider the integration time incurred by developer j for the group. The integration effort for any given link (i, z) is proportionately

distributed among the developers (i.e., proportional to each developer's involvement in the development of modules i and z). For example, suppose module i was developed by developers A and B , and module z was developed by developers A and C . Then, developer A incurs half of the effort to integrate link (i, z) while developers B and C each incur a fourth of this effort. This integration time is added to developer j 's module development time and pair formation time (when applicable) to yield the total time spent by that developer for the group.

Before we end the discussion pertaining to the effort minimization model, note that the above model would remain structurally unchanged if the module development effort, the link integration effort, and the pair formation effort were stochastic quantities and the *expected* system development effort was being optimized. This is because the objective function and the set of constraints are linear in D_{ij}^s , D_{ijl}^p , f_{jl} , λ_{iz}^s , λ_{iz}^p , and λ_{iz}^Ω .

2.3. Time Minimization

Here, we minimize the time needed to complete a project subject to the constraint that the total system development effort does not exceed a specified effort budget. Such a time minimization model may be useful to solve from the perspective of a project manager who is provided with a fixed set of resources, but needs to optimize project decisions so as to develop the system as quickly as possible. The model formulation for this problem is similar to that of the effort minimization problem except for one obvious change: the objective is to minimize the completion time of the project and the resource constraint pertains to an effort budget rather than a time deadline. For details, see Appendix A of the online supplement.²

In the effort and time minimization models, the objective functions and the constraints have some nonlinear terms (involving 0-1 variables), and hence these models cannot be directly solved by a linear integer program solver. However, it is straightforward to linearize them using standard mathematical programming techniques (Glover and Woolsey 1974). The

² An online supplement for this paper is available on the Information Systems Research website (<http://isr.pubs.informs.org/companion.html>).

additional variables and constraints required for the linearized version are provided in Appendix A of the online supplement. (See footnote 2.)

2.4. Problem Complexity

The models developed in the previous section can be solved to provide optimal module-developer assignments so that a fair comparison between the pair and solo approaches can be conducted. A practical question arises: How easy is it to solve these models? To respond to such a concern about problem complexity, we first note that these models are, in general, hard to solve. The pair programming version of the effort minimization problem is strongly NP-hard and both solo and pair versions of the time minimization problem are also strongly NP-hard. The mathematical proofs of the above claims can be found in Appendix A of the online supplement. (See footnote 2.)

Because the problem is strongly NP-hard, for problems beyond a certain size, finding an optimal solution in a reasonable amount of time can be difficult (Garey and Johnson 1979). We therefore devise a heuristic approach to obtain near-optimal solutions relatively quickly. Both the effort and the time minimization problems exhibit some structural similarity to the Quadratic Assignment Problem (QAP). In the QAP, facilities need to be assigned to locations under a cost minimizing objective that is naturally nonlinear (quadratic). Similarly, in the effort and the time minimization problems, the primary decision is to assign developers to modules with the objective of minimizing a nonlinear function. A solution technique based on a GA has been successfully applied to solve the QAP (Drezner 2003). We therefore exploit this structural similarity to devise a GA to solve our problems. The GA is briefly discussed below.

2.5. Genetic Algorithm

GAs belong to a class of heuristic optimization techniques that use randomization as well as directed search to find optimal or near-optimal solutions. The development of GAs was inspired by evolutionary processes through which life is believed to have evolved to its present forms. Goldberg (1989) has successfully applied GAs to solve many different combinatorial problems. For QAP, solutions based on

GAs have been proposed in several studies including, Ahuja et al. (2000), Drezner (2003), Fleurent and Ferland (1994), and Tate and Smith (1995).

For both the effort and the time minimization problems, we devise a GA that is based on the ideas proposed by Drezner (2003). The average performance of Drezner's algorithm has been shown to be significantly better than that of other algorithms available in the literature. More specifically, for a well-known test set consisting of 29 minimization problems, the value of the solution from Drezner's algorithm exceeded the best-known solution by only 0.037% on the average. Moreover, the algorithm is better by about a factor of 20, both with respect to the quality of the solution and the run time, than an earlier algorithm proposed by Ahuja et al. (2000).

The details of the proposed GA are provided in Appendix A of the online supplement. (See footnote 2.) To measure the performance of the GA, we compare its solution with either the optimal solution (when it can be found) or a lower bound on the optimal solution (when finding an optimal solution is not practicable). For this comparison, we consider three values for the number of modules and two values for the number of developers. Hence, there are six (3×2) problem classes. For each problem class, we generate three problem instances by varying the other model parameters.

All experiments were carried out on a Pentium IV computer (3.0 GHz, 2 GB RAM) with Windows XP as the operating system. For each problem instance, we used CPLEX (version 8.1.0) to solve the integer programming formulation of all three approaches (solo, pair, and mixed). We also used the proposed GA to solve these problem instances. The results of this comparison are reported in Table 2. CPLEX could not solve a few problem instances to optimality, but always provided a lower bound on the optimal solution. For the cases where CPLEX did not provide an optimal solution within an imposed time limit of three hours, we compare the solution from the GA with the lower bound; these results are indicated by numbers enclosed in parentheses.

From Table 2, it can be seen that our GA provides optimal or near-optimal solutions for the problem instances in the test bed. For a few problem instances

Table 2 Percentage Gap of GA Results from Optimal (or Lower Bound)

Prob. class	Number of modules	Number of developers	Percentage gap of GA solution from the optimal solution (lower bound)									Average CPU time for GA (sec.)
			Problem #1			Problem #2			Problem #3			
			Solo	Pair	Mixed	Solo	Pair	Mixed	Solo	Pair	Mixed	
1	5	3	0	0	2.19	0	0.006	3.05	0	0	2.36	1.926
2		5	0	0	0	0	0.005	6.13	0	0	2.31	3.2311
3	10	3	0.95	0	0.53	0	0	5.90	0	0	4.14	21.309
4		5	0	0	0	0.24	0.002	3.40	0	0	3.78	29.1
5	15	3	0.55	0	1.07	1.05	0	3.69	0	(0.17)	(3.95)	87.16
6		5	0.57	0	0	0.98	0	(6.51)	0.93	2.78	(5.37)	90.11

(especially, those in problem class 6), the gaps from the lower bound may appear relatively high. However, it is important to note that the gap from the optimal solution is likely to be lower than the gap from the lower bound. Note that the CPU time for our GA is considerably lower than the time CPLEX requires to solve the problem to optimality. Thus we expect the GA to provide good solutions to realistic problems in a reasonable amount of time. We therefore use the GA to solve larger problem instances considered in experiments conducted in §§3 and 4.

3. Generation of Hypotheses

In this section, we generate hypotheses relating to the optimization models developed in the previous section. These hypotheses are then explored in a set of controlled experiments in the next section. There are two methods used for generating hypotheses. We first use a greatly simplified optimization model that can be solved analytically. The main outcome of this analysis is to predict which approach (solo or pair) would perform better for a given problem, and to see when the use of a mixed approach is indicated over the better of the two pure approaches. Next, we conduct a regression experiment to see which factors have a significant impact on the effort of an approach: pair, solo, or mixed. In the next section, we conduct controlled experiments to closely examine the hypotheses generated in this section.

3.1. A Homogenous Approximation

For a given project, is it possible to perform an easy check to determine which technique (solo or pair) will incur lower effort? Next, when will the practice of mixing development regimes for the same project provide benefit? To answer these questions we propose a

greatly simplified, homogeneous approximation of a software project.

Consider a single group system where $M = \{1, 2, 3, \dots, m\}$ is the set of modules. The following assumptions are made: (1) all modules require the same development effort (d), (2) all links require the same integration effort, I (i.e., $I = \lambda_{iz}^s = \lambda_{iz}^\Omega = \lambda_{iz}^p, \forall i, z$), (3) every module is connected to the same number of other modules (r); furthermore, for every subset of module $M' \subseteq M$, the degree of every module in the subsystem created by M' is $|M'|r/m$, (4) the pair formation effort is the same for every pair of developers (f), and (5) the integration effort for the project is equally divided among all developers. To further simplify the analysis, we consider developer assignments with the property that the set of developers engaged in pair programming are divided into disjoint pairs, with each pair developing an equal number of modules. Although the assumptions behind the homogeneous approximation may rarely hold in practice, the approximation helps us tease out interesting properties of the optimization model which, in its complete form, is hopelessly intractable to analyze.

3.1.1. Analysis and Results. Because we only consider disjoint developer pairs, there are a maximum of $S/2$ disjoint pairs of developers. Our goal is to find the optimal number of disjoint developer pairs ($s^* \leq S/2$) to minimize the development effort. Assuming s pairs, the total system development effort equals $m\alpha d + sf + (1/2 - \alpha/s)mrl$. The module development work must be performed sequentially by each pair; the integration work, however, can be assumed to be done in parallel because we assume that any developer can perform any integration task. Therefore, the total system development time for the project is

$0.5(f + m\sigma d/s) + mrI(s - 2\alpha)/4s^2$. We need to choose s to minimize total effort subject to the deadline constraint. For the solo approach, the total system development effort (module development plus integration) is $md + mrI(S - \alpha)/2S$, and the total time needed is $md/S + mrI(S - \alpha)/2S^2$. We need to choose S to minimize total effort subject to the deadline constraint. It can be shown that the following effort criterion indicates that the pair approach would finish the project with less effort than the solo approach (see Appendices B and C in the online supplement [footnote 2]):

$$s^*[2s^*S^*f + m(2\sigma dS^* - 2dS^* + r\alpha I)] < 2mraIS^*.$$

The other purpose of the homogeneous approximation is to explore whether the use of a mixed approach is indicated for a given project. Here we obtain an intuitive result: the better of the two pure approaches (solo or pair) matches the performance of the mixed approach (see Appendix D in the online supplement, [footnote 2]). This result is expected; under homogeneity, optimal solutions should lie at one of the extremes (solo or pair). However, this result is an artifact of the homogeneous assumption. We therefore ask: to what extent (of heterogeneity) does the “pure = mixed” result hold? Furthermore, when the mixed approach does perform better, how substantial is the improvement? In §4, we will investigate the following two main questions that arise from the above analysis:

(a) For a given project how accurate is the prediction—based on a homogeneous approximation of the system—concerning the relative performance (based on development effort) of the solo versus the pair approach?

(b) To what extent of heterogeneity does the performance of the better of the two pure approaches (solo and pair) match the performance of the mixed approach?

3.2. Hypotheses Concerning Solo and Pair Development Efforts

We next generate a set of hypotheses that are aimed at a direct comparison—based on development effort—between the solo and pair development approaches. To generate these hypotheses, we have relied on intuition to identify some of the more obvious effects. In

addition, we use the results of a regression experiment that helps us identify more complex effects. We begin with a discussion of several specific project factors that could have a significant impact on the relative performance of the pair and solo programming approaches.

3.2.1. Discussion of Factors. The factors cluster into three categories: (1) system parameters, (2) effort parameters, and (3) project parameters.

System Parameters

Number of Modules (m) is the total number of modules in the system. Each module takes more effort to develop using pair programming.

Link density (L) is the ratio of the total number of links between modules in the system to the maximum number of links possible for that system (i.e., when each module is connected to all other modules). As the number of links in the system increases, the integration effort increases.

Effort Parameters

The *Knowledge Sharing Coefficient (α)* affects the effort savings to integrate a link from using common developers to build the modules that form the link. Because pair programming can be expected to disseminate system knowledge more effectively, the pair approach may enjoy a relative advantage over the solo approach as the knowledge sharing coefficient is increased.

Pair Effort Factors (f, σ) only affect the pair programming approach. As mentioned earlier, pair formation effort—also referred to as pair jelling effort—is the initial adjustment effort required to transition from solitary to collaborative programming. Another factor related to pair programming, the pair development overhead, is the additional effort needed by a pair of developers to develop a module when compared to the effort incurred by an individual developer to build the same module.

Project Parameters

Project Deadline (T) is the total time available to complete the project. As the project deadline tightens, more parallel work must occur to complete the system within schedule. As more work is performed in parallel, the knowledge dissemination advantage

of pair programming should reduce. Another project related parameter is the *number of developers* (S) available for the project. For a given project, the number of developers and the project deadline are clearly related: more developers would typically be required to complete a given project in less time. Rather than varying both the project completion time and the team size, we keep the team size constant at seven developers and generate project deadlines that are tight or relaxed in relation to this fixed team size.

Team Expertise (n_e) is the number of expert developers in the team. This factor influences system development effort because it is assumed that expert developers can perform the same task with less effort.

The above factors are varied to create realistic project conditions to explore their effects. Real software systems, however, can vary widely in many aspects such as structure, scope, complexity, etc., and in a numerical experiment, all nuances of a real software system are difficult to capture. To generate the data for our experiments we used artificial systems, but anchored the characteristics of these systems in some real context—a billing application being implemented using pair programming methods at a large telecommunications software company. The details of the billing system and the numerical factor values used in the experiment are described in the appendix (see Appendix E in the online supplement, [footnote 2]).

We use the following model,

$$\text{Effort}(E) = B_0 + B_1X + B_2\alpha + B_3f + B_4\sigma + B_5m + B_6n_e + B_7L + B_8T + \text{Interaction Terms.} \quad (1)$$

The model has one indicator variable ($X = 1$ for pair, $= 0$ for solo), six quantitative variables (α , f , σ , m , n_e , and L), and one ordinal variable (T), each with two levels (low and high). The estimated regression model is presented in Appendix E in the online supplement (see footnote 2). The parameters are estimated using the Ordinary Least Squares approximation in SAS 9.1. Firstly, a reduced form of the model is obtained using *stepwise regression*. In the reduced model, only terms with p -value below 0.05 are considered significant. To eliminate multicollinearity, we use a partial orthogonalization of interaction terms (Burrill 1997, Yu 2000). Using this method we were able to derive parameter estimates of all explanatory variables with

variation inflation factors less than 6, indicating that after orthogonalization, the multicollinearity problem had been resolved (Kleinbaum et al. 1998, p. 210). For robustness, we also verified that the condition indices and tolerances associated with the parameter estimates demonstrated the absence of multicollinearity among the explanatory variables (see Appendix E in the online supplement [footnote 2]). In addition, a well-known goodness-of-fit test for normality (Kolmogorov-Smirnov) was used to verify that the error term in (1) is normally distributed with a mean of zero.

3.2.2. Hypotheses. We next present seven statements that summarize the impact of the various factors on the effort of the pair and solo approaches. These statements are further examined in a controlled experiment in §4. The first four statements below list conditions that put the pair approach at a disadvantage followed by another three statements that list conditions which favor the use of the pair development approach. All statements below use total system development effort as the basis of comparison.

HYPOTHESIS 1. *Increasing the pair development overhead always puts the pair approach at a disadvantage relative to the solo approach.*

This hypothesis is easy to motivate: Because increasing the pair development overhead only worsens the performance of the pair approach, this approach should be put at a relative disadvantage when the pair development overhead is increased.

HYPOTHESIS 2. *The relative disadvantage of increasing the pair development overhead for the pair approach is magnified for a system with more modules, and with high values for the knowledge sharing coefficient or the project deadline.*

It is reasonable to expect the relative disadvantage to increase with more modules in the system because each module requires relatively more effort to be developed as a pair. The impact of the knowledge sharing coefficient on the relative disadvantage is more subtle, however. As the pair development overhead is increased, more development effort is needed by the pair approach than by the solo approach. For a fixed project deadline, the only way to accomplish this additional development effort is for the

pair approach to use more pairs to complete the project. Hence, knowledge sharing could suffer and the relative impact of reduced knowledge sharing should be more at higher values of the knowledge sharing coefficient. Finally, the manner in which the project deadline affects the relative disadvantage is also related to the number of pairs used to complete the project. At low values of the project deadline, relatively more pairs are already necessary to complete the project and hence increasing the pair development overhead should not have much impact on the number of pairs used. On the other hand, at high values of the project deadline, the pair approach is able to use relatively fewer pairs and hence benefit from its superior knowledge sharing ability. This ability is restricted by increasing the pair development overhead and hence, the relative disadvantage (of increasing the pair development overhead) should be felt more when the value of the project deadline is high rather than low.

HYPOTHESIS 3. *Increasing the pair formation effort has a negative impact on the pair approach relative to the solo approach only when the project deadline is relatively tight (i.e., a low value of the deadline).*

The forces behind this hypothesis are similar to the ones discussed above. Since the pair formation effort applies to each pair being formed, the impact of increasing the pair formation effort should depend on the number of pairs being used to complete the project. For a project with a tight deadline, it is necessary to use relatively more pairs to complete the project before the specified deadline. Hence, the impact should be more for a project with a tight deadline.

HYPOTHESIS 4. *Increasing the number of modules puts the pair approach at a relative disadvantage when the pair development overhead is high and the link density is low.*

The intuition behind the first part of the above hypothesis is clear and has been discussed earlier. When the number of modules is increased at high values of the link density, the number of links also increases rapidly and this should favor the pair approach, thus compensating to a certain extent, the increase in the number of modules. Hence the relative disadvantage should be felt more when the link density is low.

HYPOTHESIS 5. *Increasing the knowledge sharing coefficient benefits the pair development approach relative to the solo approach. However, this relative advantage is weakened when the pair development overhead is high.*

The first statement above is intuitive and results from the superior knowledge sharing ability of the pair approach. The second statement (concerning the relative advantage being weakened) has been discussed earlier: when the pair development overhead is high, more pairs are necessary to complete the project and this restricts the advantage of the pair approach to exploit knowledge sharing. Hence, the relative advantage of increasing the knowledge sharing coefficient can be expected to weaken when the pair development overhead is high.

HYPOTHESIS 6. *Increasing the link density in a system benefits the pair development approach relative to the solo approach. This relative advantage increases as the number of modules in the system increases.*

The first statement is intuitive. As higher values of link density, the integration effort in the project is relatively higher. Generally speaking, the relative strength of the pair approach has to do with its lower integration effort. Hence, we should expect that increasing the link density should favor the pair approach. The second statement is also easy to explain. With more modules in the system, increasing the link density has a greater impact on the number of links in the system. Hence, the relative advantage should increase.

HYPOTHESIS 7. *Relaxing (i.e., increasing) the project deadline puts the pair approach at a relative advantage over the solo approach. This relative advantage increases for high values of the pair formation effort and reduces for high values of the pair development overhead.*

The first statement can be explained by the fact that when the project deadline is lenient, the pair approach is able to form relatively fewer pairs, and hence exploit its knowledge sharing advantage. The relative advantage is magnified at high values of the pair formation effort because increasing the project deadline not only increases knowledge sharing (by forming fewer pairs) but, more directly, reduces the total pair formation effort. The last effect above concerns the fact that at high values of the pair development overhead, relatively more pairs need to be created to complete the project within the specified deadline. Hence,

the relative advantage of the pair approach (resulting from superior knowledge sharing) should diminish.

4. Controlled Experiments

The empirical and analytical methods used in the previous section provided us with several hypotheses that we explore in depth in this section. These hypotheses need further exploration because neither the regression model nor the homogeneous approximation was sufficiently detailed. The results from the homogeneous approximation are questionable because of the strong homogeneity assumptions made to obtain these results. Also, for practical reasons, the regression model used only two levels for each factor. Two level variations do not provide much sense of the nature of the relationship between a particular factor and the outcome variable of interest (effort or time). In these experiments, we isolate a factor of interest and vary it in small increments from a low value to a high value. The other factors are held constant while the factor of interest is being varied. To permit some investigation into factor interactions, an experiment is repeated for different values of the interacting factor of interest.

To begin, we evaluate the predictive ability of the effort criterion developed in the previous section over a wide range of problems. To summarize, the effort criterion correctly predicted for about 83.59% of the 128 instances to which it was applied. Despite the homogeneity assumptions, the various influencing factors appear to “average” out at least as far as the *ordinal* ranking of the pair and solo approaches is concerned. Thus, for effort minimization problems, the criterion should prove to be quite useful to provide a rough guideline on when the pair approach should be used.

4.1. Pure Versus Mixed

As regards the issue of “pure versus mixed,” we first use the data generated for the regression experiments to test whether mixing development approaches in the same project can lower the development effort. The one-tailed *t*-test shows that it is not necessary to mix the solo and the pair development practices in the same project. We now investigate this result in greater detail using the controlled experiments.

The controlled experiments allow us to make two observations. The first observation is that the “pure = mixed” result continues to hold as long as the groups share the same average characteristics (e.g., the average number of links emanating from given module). Within a group, differences between modules (e.g., some modules being more connected than others) do not indicate the use of a mixed approach. Thus, “inter-group” variation is more indicative of the use of a mixed approach than “intra-group” variation.

Our inferences above draw from two separate experiments. In the first experiment, we let intra-group variation increase, but keep the mean characteristics of the groups constant. For example, when the connection probability is 0.1, all groups have the characteristic that any pair of modules in a group is connected with a probability of 0.1. The connection “event” between modules is a Bernoulli variable with a variance of $0.1 * (0.9)$. As the connection probability increases from 0.1 to 0.5, it is clear that the intra-group variation should increase. However, this increase does not seem to affect the pure versus mixed choice; a pure approach always matches the performance of the mixed approach.

In the second experiment, we create differences in the mean characteristics of the groups; the groups are different in terms of the mean connection probability. In this experiment, the number of links is determined based on the connection probability between the modules within a group and the connection probability between the modules across groups. Because the number of links within a group is usually much higher than the number of links across any two groups, the connection probability between the modules across groups is fixed at 0.02 and the connection probability between the modules in the first group is fixed at 0.95; for the other groups, this probability is varied systematically as shown in Figure 1. The other parameter values used in the second experiment are as follows: Number of Groups = 3, Number of Modules = 20, Pair Development Overhead = 1.25, Project Deadline = 3.4 Weeks, Number of Developers = 7, Number of Expert Developers = Number of Novice Developers = 2. Finally, the mean knowledge sharing coefficient was set at 0.45 and the mean pair formation effort was chosen to be 0.15 person-weeks; the variation in these parameters was generated as described in

Figure 1 Comparison of Pure Versus Mixed Approaches

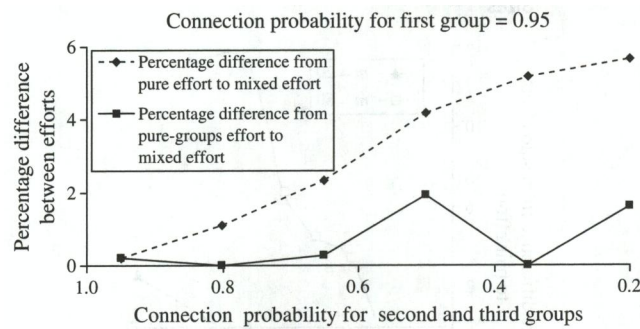


Table 3 Base Parameter Values for Examining the Factor Effects

G	M	Mean knowledge sharing coefficient	Mean pair formation effort	L	σ	S
6	40	0.15	0.30 person-weeks	0.11	1.15	7

Appendix E.2 of the online supplement (see footnote 2). The results of the second experiment are shown in Figure 1. Note that the percentage difference from pure effort to mixed effort is calculated as:

$$\text{Percentage Difference from Pure Effort to Mixed Effort} = \frac{\text{Pure Effort} - \text{Mixed Effort}}{\text{Pure Effort}} \times 100.$$

In Figure 1, the dashed line shows the improvement of the mixed approach over the better of the two pure approaches. Here, the mixed approach is always indicated as the development method of choice. Our second observation comes from a closer look at situations when the use of a mixed approach is indicated. Here, we find that most of the benefits provided by a mixed approach can be achieved by using a special case of the mixed approach (referred to as “Pure Groups”). In this approach mixing regimes is not permitted *within* a group, but the regimes can be different *across* groups. This effect can be seen in Figure 1, where the percentage difference (the solid line) from pure-groups effort to mixed effort is less than 2%.

4.2. Factor Effects

The base parameter values are given in Table 3. In addition, the number of expert developers was set equal to the number of novice developers ($=2$). For each experiment, all parameter values are fixed at

their base values except for the parameters being varied. Finally, we calculate the minimum feasible time needed to complete each problem instance in an experiment using solo and pair programming, and set the project deadline at the maximum of these values for that experiment. This is done to guarantee that each problem instance in these experiments has at least one feasible solution. Appendix E.2 of the online supplement (see footnote 2) describes the process of generating realistic systems for a given link density (L) and also describes how values of the link integration effort for each link were chosen to depend on the expertise level of the developers that were common to the modules connected by the link.

4.2.1. Impact of Pair Development Overhead and Pair Formation Effort. As predicted by the regression experiment, we find that increasing the pair development overhead puts the pair approach at a relative disadvantage with respect to the solo approach (Figure 2). Moreover, the negative impact of increasing the pair development overhead on the pair approach is clearly seen to magnify when there are more modules in the system.

Also as indicated by the regression experiment, we find that increasing the pair formation effort typically hurts the pair approach. In addition, we find that this effect is more pronounced when the number of modules is high (Figure 3). This can be explained as follows. For a given project deadline, when the number of modules is small the pair approach is able to use relatively fewer distinct pairs and complete the project in time. Thus, the overall development

Figure 2 Relative Impact of Pair Development Overhead

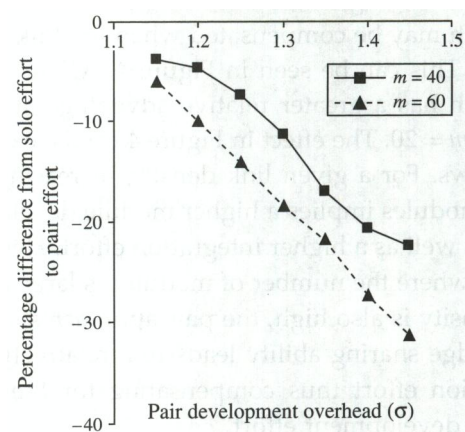
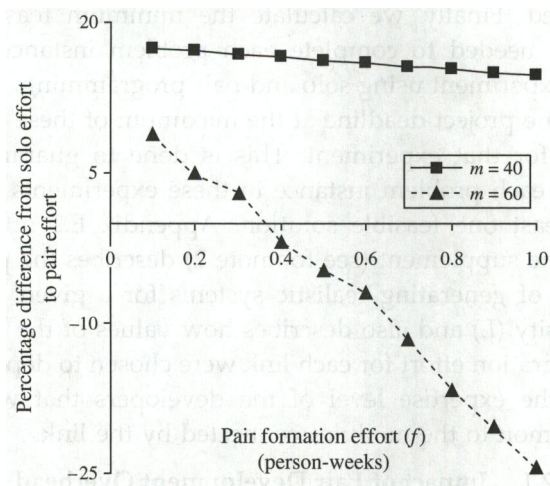
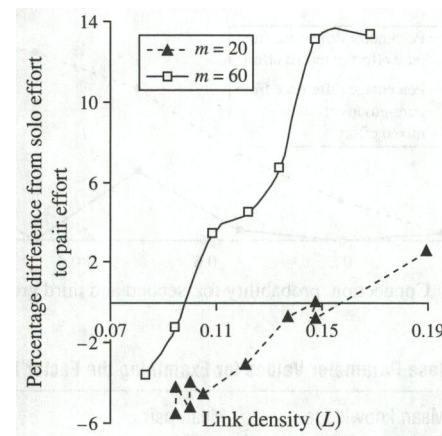


Figure 3 Relative Impact of Pair Formation Effort

effort is less sensitive to changes in the pair formation effort. In contrast, when number of modules is high more distinct pairs need to be formed to complete the project by the same deadline. Having more distinct pairs amplifies the impact of a higher pair formation effort. This effect can be clearly seen in Figure 3, where the rate at which the pair approach is hurt by increasing f is higher for $m=60$ as compared to that for $m=40$.

The pair formation effort and the pair development overhead are not applicable to the solo approach; hence an increase in these factors should adversely impact only the pair approach. As seen in Figures 2 and 3, we find that increasing the number of modules hurts the pair approach more when the pair development overhead or the pair formation effort is high.

4.2.2. Impact of Link Density. The negative impact of increasing the number of modules on the pair approach may be compensated when the link density is high. This can be seen in Figure 4, where the pair approach has a greater relative advantage at $m=60$ than at $m=20$. The effect in Figure 4 can be explained as follows. For a given link density, increasing number of modules implies a higher module development effort as well as a higher integration effort. Hence in a system where the number of modules is large and the link density is also high, the pair approach's superior knowledge sharing ability leads to a relatively lower integration effort thus compensating for the higher module development effort.

Figure 4 Relative Effort Impact of Link Density at Different System Sizes

4.2.3. Impact of Knowledge Sharing Coefficient.

An interesting result (not identified by the regression experiments) is that the pair approach is better at leveraging expertise within the development team (Figure 5). Unlike the solo approach, a single expert can be paired with many different (typically average or novice) partners, thus spreading the knowledge and skills of this expert across many tasks. The knowledge sharing coefficient plays a secondary, but important role here. At higher levels of the knowledge sharing coefficient, the pair approach further exploits the presence of more expertise within the team.

Consistent with the regression model, in Figure 6, we find that increasing the link density favors the pair approach over the solo approach. Increasing the link density for a given number of modules increases only the system integration effort. The increased system

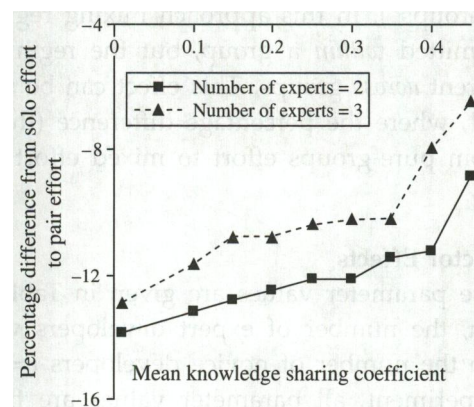
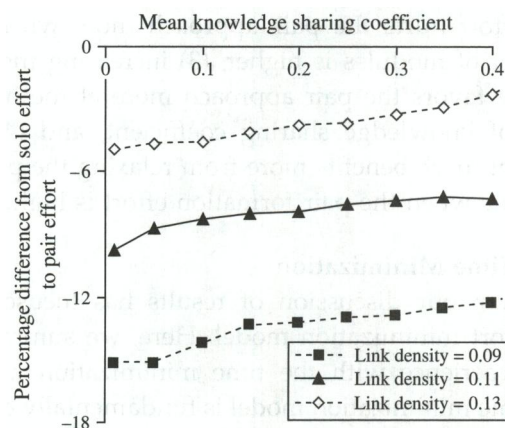
Figure 5 Relative Effort Impact of Knowledge Sharing Coefficient at Different Levels of Expertise

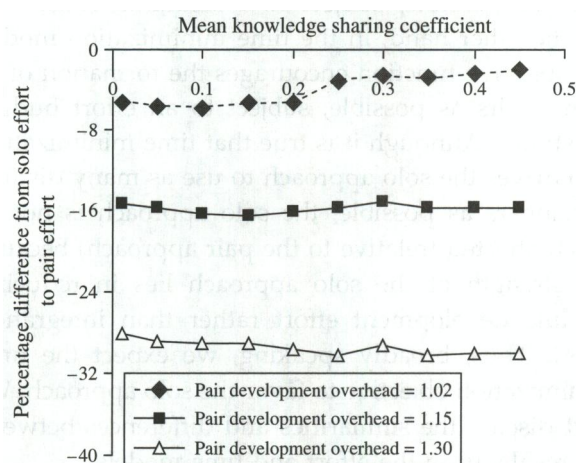
Figure 6 Relative Effort Impact of Knowledge Sharing Coefficient at Different Link Densities



integration effort helps the pair approach gain a relative advantage over the solo approach. In addition, it can be observed that this advantage amplifies (albeit slightly) at higher levels of the knowledge sharing coefficient.

Increasing the knowledge sharing coefficient typically benefits the pair approach more than the solo approach. However, this relative advantage is weakened (and can reverse to become a disadvantage) when the pair development overhead is high (Figure 7). This reduction in relative benefit occurs because at higher levels of the pair development overhead, more pairs are needed to complete the project in time; this limits the ability of pair approach to leverage knowledge sharing.

Figure 7 Relative Effort Impact of Knowledge Sharing Coefficient at Different Values of Pair Development Overhead



4.2.4. Impact of Project Deadline. The project deadline has an important impact on the relative performance of the pair and solo approaches. In line with our regression experiment, we find that both approaches benefit as the project deadline increases, but the pair approach benefits more. Also, the magnitude of this relative advantage increases when the pair formation effort is higher (Figure 8). At first glance, this effect is surprising, because the pair formation effort is a drawback of the pair approach and is not applicable to the solo approach. The effect can be explained as follows. As T is increased, the project can be completed with fewer distinct pairs (Figure 9) and the superior knowledge sharing ability of the pair approach can be better leveraged. Increasing the deadline not only reduces the integration effort but, in the case of the pair approach, it also reduces the total pair formation effort. This, in turn, translates into greater savings in the overall effort when the pair formation effort is high. Figure 8 illustrates two interesting regions; in the first region ($T < 10.5$), when the project deadline is increased there is a gradual increase in the relative advantage of the pair approach. This can be attributed to reduced pair splitting and hence lower integration effort. In contrast, in the second region ($T > 10.5$), there are dramatic benefits to the pair approach from increasing the project deadline. This effect can be attributed to a decrease in the total number of pairs formed. However, once the project deadline increases to (and beyond) the point where only one pair needs to be formed (Figure 9),

Figure 8 Relative Effort Impact of Desired Project Duration at Different Values of Pair Formation Effort

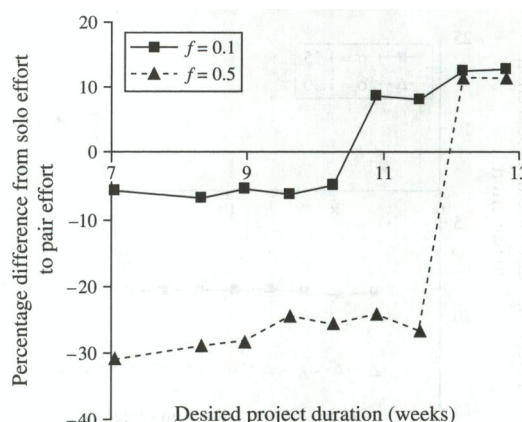
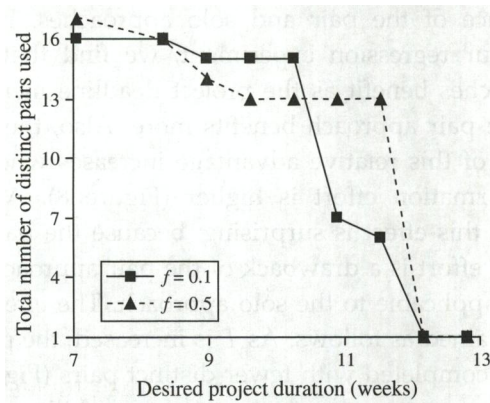
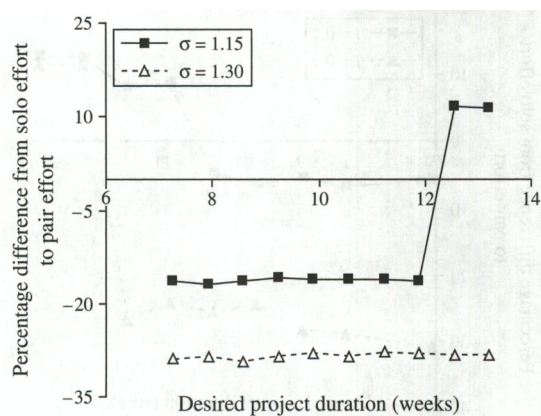


Figure 9 Relative Impact of Desired Project Duration on Number of Distinct Pairs at Different Values of Pair Formation Effort

the pair formation effort has minimal impact (i.e., the effort curves in Figure 8 for $f = 0.1$ and $f = 0.5$ almost merge for large T).

Finally, consistent with our regression analysis, we find that increasing the pair development overhead reduces the relative benefit derived by the pair approach from relaxing the project deadline (Figure 10). This is due to the fact that when the pair development overhead is high, even under a relaxed project deadline it is difficult for the pair approach to reduce the total number of pairs needed to complete the project.

In summary, the controlled experiments reinforce and shed deeper insights into the hypotheses generated by the regression experiments. Additionally, we find the following new factor effects that were not revealed by the regression experiments: (1) the

Figure 10 Relative Effort Impact of Desired Project Duration at Different Values of Pair Development Overhead

pair approach is better at leveraging expertise within the development team, (2) increasing the pair formation effort hurts the pair approach more when the number of modules is higher, (3) increasing the link density favors the pair approach more at the higher level of knowledge sharing coefficient, and (4) the pair approach benefits more from relaxing the project deadline when the pair formation effort is high.

4.3. Time Minimization

Thus far, our discussion of results has focused on the effort minimization model. Here, we summarize our experience with the time minimization model. The time minimization model is fundamentally different from the effort minimization model. The attempt in the effort minimization model is to maximize the commonality between developers as much as possible. However, because the use of common developers creates sequential paths in the project, the extent to which common developers can be used is limited by the deadline constraint. The attempt in the time minimization model, on the other hand, is to create as much parallel work in the project as possible since doing so typically reduces the time taken to complete the project. However, more parallel work comes at the cost of increased effort during integration since commonality gets sacrificed in the interest of saving time.

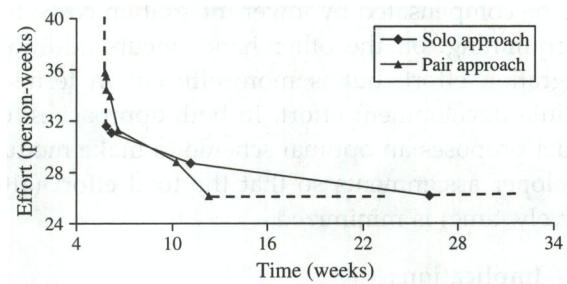
Our experience with the effort minimization model has revealed that one key advantage of the pair approach is its superior ability to leverage knowledge sharing and hence reduce integration effort. This ability of the pair approach is endorsed by an objective function that encourages the formation of as few pairs as possible, subject to a deadline constraint. On the other hand, in the time minimization model, the objective function encourages the formation of as many pairs as possible, subject to an effort budget constraint. Although it is true that time minimization also drives the solo approach to use as many distinct developers as possible, the solo approach is not as much affected (relative to the pair approach) because the strength of the solo approach lies in reducing module development effort rather than integration effort. Thus, broadly speaking, we expect the time minimization objective to favor the solo approach. We next discuss the similarities and differences between the results from the effort and time models.

4.3.1. Similarities and Differences: Effort versus Time Minimization. In the time minimization model, increasing the pair development overhead (or pair formation effort) has an adverse impact on the pair approach. This impact is reduced at higher levels of the effort budget, because the pair approach is able to absorb the higher module development overhead (or pair formation effort) and still perform sufficient parallel work. In addition, similar to the effort minimization model, we find that the negative impact of increasing the pair development overhead is magnified when the number of modules is high.

In the time minimization model, increasing the link density puts the pair approach at a relative *disadvantage*; however, this effect is only observed for a relaxed effort budget. This effect is different from the one observed in the effort minimization model where increasing link density always favors the pair approach relative to the solo approach. This finding is consistent with our expectation that there is a conflict between the objective of doing more parallel work and sharing knowledge. Increasing the link density increases the total effort requirements for the project. At the same time, as explained above, the time minimization objective hinders the ability of the pair approach to use knowledge sharing as a means to reduce effort. The pair approach is therefore driven to use less productive solutions. However, when the effort budget is sufficiently tight, the problem effectively becomes one of minimizing the project effort to meet the budget constraint. Under these conditions (i.e., when the effort constraint drives the solution), increasing link density always benefits the pair approach more than the solo approach.

For the time minimization model, we observe that increasing the number of modules could either have a beneficial or harmful impact on the pair approach relative to the solo approach. This finding is similar to the one observed for the effort minimization model. Increasing the number of modules increases both the module development effort as well as the integration effort. When the pair development overhead is low and the link density is high, the inherent ability of the pair approach to reduce integration effort allows it to absorb the increased module development effort better than the solo approach. The reverse is true when the pair development overhead is high and the link density is low.

Figure 11 A Pareto Frontier for Bi-Criteria Decision-Making



4.4. Pareto Frontier

In practice, a project manager may be interested in *both* the effort and time objectives. The problem then is to choose a development approach that, based on the relative importance of time and effort, achieves the best trade-off between these objectives. To make such a trade-off, the effort and time minimization models can be used to develop a Pareto frontier to facilitate bi-criteria decision-making. The frontier, drawn for a given set of project parameters and development method, is a set of points (E^*, T^*) . A point on this frontier implies that E^* is the minimum effort with which the project can be completed with a time constraint of T^* , and T^* is the minimum time needed to complete the project with an effort budget of E^* . Figure 11 shows the Pareto frontiers drawn for a project for the solo and pair approaches. The frontiers clearly show that the solo approach dominates for a tight time budget, whereas the reverse is true for a tight effort budget. The dotted portions in each frontier represent values of the time or budget constraint where the shadow price of relaxing the deadline constraint is zero. Although the exact points on these frontiers depend on the project parameters, the nature of the trade-off in Figure 11 is representative of the inherent strengths and weaknesses of the solo and pair approaches.

5. Discussion and Conclusions

This study is among the first of its kind to quantitatively explore the various pros and cons of pair programming—a novel approach to software development. In essence, the model developed in this paper is a theory of team work in software development. The model highlights the trade-off between module development and system integration effort. In pair

programming, the extra module development effort may be compensated by lower integration costs. Solo programming, on the other hand, incurs additional integration effort, but is more efficient in terms of module development effort. In both approaches, our model proposes an optimal scheme to make module-developer assignments so that the total effort (alternatively, time) is minimized.

5.1. Implications

We have shown analytically, under special homogeneity assumptions, that the performance of the better of the two pure approaches matches the performance of the mixed approach. Experimentally, we show that this result also extends to the case of a heterogeneous project where groups share the same average characteristics (such as number of modules, connection density, etc.). These results confirm our intuition that the “local neighborhood” of a module is the most important determinant (albeit not the only one) of the choice of the development approach (solo or pair). When the modules in a group are homogeneous, they share a similar local neighborhood in terms of the number of connected modules and the complexities of these modules. Hence, the optimal development approach for each module stays the same within the same group. The analytical result in the online supplement can also be interpreted in a similar way. There, under assumptions of strict homogeneity and a system consisting of a single group of modules, we were able to prove that a strictly mixed approach is always dominated by a pure approach. In mathematical terms, this result means that a corner solution is optimal, i.e., the proportion of modules developed using the pair approach (ρ , a decision variable) is either 0 or 1, but never in between. Furthermore, if the groups are similar to each other, this result extends to the whole system. Our experiments also showed that the mixed approach indeed outperforms the best pure approach when groups are heterogeneous in their mean characteristics. Interestingly, even for such cases we found that mixing regimes is typically not needed *within* a group (when the modules in a group are homogeneous), although different regimes may be chosen *across* groups.

Our analysis shows that the comparative advantage of the pair approach lies in reduced integration effort due to better knowledge sharing, whereas

the comparative advantage of solo approach lies in the reduced development effort (and no pair formation effort). In addition, we have shown that the solo approach is better suited for minimizing the project completion time, whereas the pair approach is more appropriate for minimizing development effort. Together these results indicate that pair programming may be better suited for novel projects where completion time is less of a concern, but there is a strong need for every team member to understand the nuances of the development task.

We have mentioned earlier that pair programming is a tacit or implicit way to share knowledge—it is a “learning by doing” approach. Solo programming projects also require knowledge sharing; however, the sharing of knowledge in such projects occurs through integration meetings, code “walk-throughs,” and other more explicit methods. One subtle aspect of pair programming revealed here is that pair splitting is often necessary to fully leverage the benefits of the approach. Thus, pair programming is not simply a technique where a pair of developers (rather than a single developer) becomes the new lowest unit of programming capacity. Otherwise, it would be sufficient to create a fixed number of developer pairs for use in a project. Many situations, however, require that pairs split during the project.

5.2. Limitations

Our model of pair programming considers pair formation effort for a single project. Initial observations suggest that once a pair “jells,” it may not be necessary to re-invest this effort when the pair is used in other projects. At the very least, the re-formation effort has been informally observed to be much lower than the first-time effort. In addition, there is evidence that suggests that developers may benefit from earlier experiences with pair formation: When a new pair is formed with members who have participated in earlier pair-programming tasks, the pair-formation effort incurred could be significantly lower. These observations indicate that organizations may find the pair formation effort to be more of a fixed effort that can be amortized across several projects and only a small amount of “pair-maintenance” effort may be incurred within a given project. If such a view of pair formation is indeed upheld, then the balance will further

tilt toward the use of pair programming for software development—implying that the results in this paper at low values of the pair formation effort may be more representative.

A limitation of the model presented here is that it emphasizes total system development effort (and system development time) as the only criteria to compare the solo and pair programming approaches. However, there are other aspects of pair programming that could be beneficial. For example, pair programming, through pair formation and pair splitting, may have social and psychological benefits. If applied effectively, it could reduce the tedium of software development and improve developer morale.

On the other hand, there are potential “incentive” problems in implementing pair programming that this study did not consider; specifically, what effects does joint code ownership have on the quality of the output? Unless incentive mechanisms are correctly in place, it is possible that there is some extent of “free-riding” when a developer pair jointly develops code. This may be one of the reasons why pairs with nonsymmetric experience and abilities are formed, e.g., partnering a junior programmer with a senior architect. In such partnerships, the two individuals contribute in different ways, e.g., the senior architect provides high-level architectural expertise and evaluates the code for quality and functionality, whereas the junior programmer contributes by working out details and actually writing the code. If two developers of equal skill and experience are paired, it is possible that incentive issues become perverse and free-riding becomes a problem.

5.3. Conclusions

In this paper, we developed an analytical model to optimize module-developer assignments in software development with the objective of minimizing the total system development effort (or total development time). The main purpose of the study was to compare the optimal assignments under two regimes: solo programming and pair programming. Using a combination of experimental and analytical results we were able to provide several recommendations for software development practice. A useful result of our analysis was that there is not much support for simultaneously operating in both the regimes,

solo as well as pair, within a group. However, when the groups are sufficiently heterogeneous in their average characteristics, it may sometimes be optimal to choose different regimes across groups for the same software project. We proposed an accurate means of predicting the superior approach for a given project based upon average project parameters values (module complexity, connectivity, and pair formation effort) that should be relatively easy to estimate for a given project. Because the prediction technique was based on strict homogeneity assumptions, we ran extensive numerical experiments (in which these assumptions were relaxed) to gain insights into conditions in which the solo or the pair approach would be preferred. When the knowledge sharing between the developers is efficient or when the functional dependence between the system modules is high, the pair approach is preferable over the solo approach. The pair approach also appears to be better than the solo approach at distributing scarce expertise across more development tasks (by forming more pairs with expert developers). If the pair development overhead or the pair formation effort is high, the solo approach is more suitable. Projects with a tight deadline are less suited for pair programming.

References

- Ahuja, R. K., J. B. Orlin, A. Tiwari. 2000. A descent genetic algorithm for the quadratic assignment problem. *Comput. Oper. Res.* 27 917–934.
- Ambler, S. W. 2002. *Agile Modeling: Effective Practices for eXtreme Programming and the Unified Process*. John Wiley & Sons, Inc., New York.
- Astels, D., G. Miller, M. Novak. 2002. *A Practical Guide to Extreme Programming*. Prentice Hall, Upper Saddle River, NJ.
- Baecker, R., J. Grudin, W. Buxton, S. Greenberg. 1995. *Readings in Human-Computer Interaction: Toward the Year 2000*. Morgan Kaufmann Publishers, Inc., San Francisco.
- Beck, K. 2000. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, Boston.
- Beck, K., M. Fowler. 2001. *Planning Extreme Programming*. Addison-Wesley, Boston.
- Benedicenti, L., R. Paranjape. 2001. Using extreme programming for knowledge transfer. *Proc. 2nd Internat. Conf. eXtreme Programming and Agile Processes in Software Engrg. (XP2001)*, Villasimius, Sardinia, Italy.
- Brown, J. S., P. Duguid. 1991. Organizational learning and communities of practice: Toward a unified view of working, learning and innovation. *Organ. Sci.* 2(1) 40–57.
- Burrill, D. 1997. Modeling and interpreting interactions in multiple regression. Available at <http://www.minitab.com/>.

- Cockburn, A., J. Highsmith. 2001. Agile software development: The people factor. *Computer* (November) 131–133.
- Cockburn, A., L. Williams. 2000. The costs and benefits of pair programming. *Proc. 1st Internat. Conf. eXtreme Programming and Flexible Processes in Software Engrg. (XP2000)*. Cagliari, Sardinia, Italy.
- Curtis, B., H. Krasner, N. Iscoe. 1988. A field study of the software design process for large systems. *Comm. ACM* 31(11) 1268–1287.
- Dingsoyr, T. 2002. Knowledge management in medium-sized software consulting companies. Ph.D. thesis, Department of Computer and Information Science, Norwegian University of Science and Technology, Trondheim, Norway.
- Drezner, Z. 2003. A new genetic algorithm for the quadratic assignment problem. *Inform. J. Comput.* 15(3) 320–330.
- Erdogmus, H., L. Williams. 2003. The economics of software development by pair programmers. *Engrg. Economist* 48(4) 283–319.
- Fleurent, C., J. A. Ferland. 1994. Genetic hybrids for the quadratic assignment problem. P. Pardalos, H. Wolkowicz, eds. *Quadratic Assignment and Related Problems, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, 16 173–187.
- Garey, M. R., D. S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co., New York.
- Glover, F., E. Woolsey. 1974. Converting the 0-1 polynomial programming problem to a 0-1 linear program. *Oper. Res.* 22(1) 180–182.
- Goldberg, D. E. 1989. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, Wokingham, UK.
- Grunbacher, P., C. Hofer. 2003. Complementing XP with requirements negotiation. *Proc. 4th Internat. Conf. eXtreme Programming and Agile Processes in Software Engrg. (XP2003)*. Genova, Italy.
- Hailpern, B., P. Santhanam. 2002. Software debugging, testing and verification. *IBM Systems J.: Software Testing and Verification* 41(1).
- Highsmith, J., A. Cockburn. 2001. Agile software development: The business of innovation. *IEEE Comput.* (September) 120–122.
- Ippolito, B., E. Murman. 2001. Improving the software upgrade value stream. Lean Aerospace Initiative (LAI) Monograph, Working Paper ESD-WP-2002-02, Engineering Systems Division, Massachusetts Institute of Technology.
- Kleinbaum, D. G., L. L. Kupper, K. E. Muller. 1998. *Applied regression analysis and other multivariate methods*. PWS-Kent, Boston.
- Kuppuswami, S., K. Vivekandanam, P. Ramaswamy, P. Rodrigues. 2003. The effects of individual XP practices on software development effort. *ACM SIGSOFT Software Engrg. Notes* 28(6, November) 6–13.
- Lui, K. M., K. C. C. Chan. 2004. A cognitive model for solo programming and pair programming. *Proc. 3rd IEEE Internat. Conf. Cognitive Informatics (ICCI'04)*, Victoria, Canada. IEEE Computer Society, Washington D.C. 94–102.
- Meixell, M. J., M. Nunez, A. Talalayevsky. 2006. Activity structures in a project-based environment: A coordination theory perspective. *IEEE Trans. Engrg. Management* 53(2, May) 285–296.
- Nawrocki, J., A. Wojciechowski. 2001. Experimental evaluation of pair programming. *Proc. 12th Eur. Software Control and Metrics Conf., London, UK*, Shaker Publishing BV, Maastricht, The Netherlands. 269–276.
- Nielsen, J. 1993. *Usability Engineering*. Academic Press, Boston.
- Nosek, J. T. 1998. The cast for collaborative programming. *Comm. ACM* 41(3) 105–108.
- Pressman, R. 1992. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, New York.
- Shukla, A. 2002. Pair programming and the factors affecting Brook's law. Master's thesis, North Carolina State University, Raleigh, NC.
- Succi, G., M. Stefanovic, W. Pedrycz. 2001. Quantitative assessment of extreme programming practices. *Proc. 2001 Canadian Conf. Electrical and Comput. Engrg. Toronto*. IEEE, Washington, D.C. 81–86.
- Tate, D. M., A. E. Smith. 1995. A genetic approach to the quadratic assignment problem. *Comput. Oper. Res.* 22 73–83.
- Waltz, D., B. Curtis, J. Elam. 1993. Inside a software design team: Knowledge acquisition, sharing and integration. *Comm. ACM* 36(10) 62–77.
- Wells, D. 2003. Extreme programming: A gentle introduction. Available at <http://extremeprogramming.org>, January 26.
- Williams, L., R. Kessler. 2003. *Pair Programming Illuminated*. Addison-Wesley.
- Williams, L., R. Kessler, W. Cunningham, R. Jeffries. 2000. Strengthening the case for pair-programming. *IEEE Software* 17(4, July/August) 19–25.
- Wood, A., W. Kleb. 2002. Extreme programming in a research environment. Technical report, NASA Langley Research Center, Hampton, VA.
- Yu, C. H. 2000. An overview of remedial tools for collinearity in SAS. *Proc. 2000 Western Users of SAS Software Conf., Scottsdale, AZ*, SAS Software, Cary, NC, 196–201.